

## Chapter 30: Binary Trees in Java

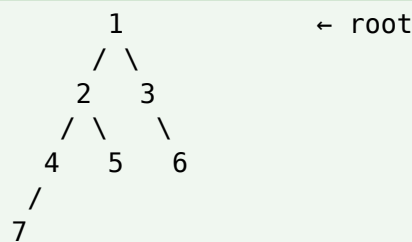
Personal Notes

Arrays, strings, and HashMaps are LINEAR — one element after another. Trees are HIERARCHICAL — data branching out from a root. They're how databases organize indexes, how filesystems organize folders, how parsers represent expressions, how routing tables work. They're also the single most-asked topic in coding interviews.

Binary trees are the simplest form: each node has at most two children. They're the foundation for binary search trees, heaps, expression trees, tries, and (eventually) graphs. If you genuinely understand binary trees, the rest of DSA gets dramatically easier.

### 1. What is a Tree?

A tree is a collection of NODES connected by edges, forming a hierarchical structure with no cycles. Each tree has exactly ONE root (the topmost node), and every other node descends from it.



Node: each circle (1, 2, 3, 4, 5, 6, 7)  
Edge: each line connecting two nodes  
Root: the topmost node (1)  
Leaf: a node with no children (5, 6, 7)  
Parent: the node above another (2 is parent of 4 and 5)  
Child: the node below another (4 is child of 2)  
Sibling: nodes sharing a parent (4 and 5)

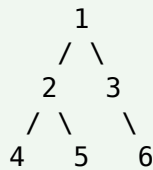
### Key Terms

| Term   | Meaning                          |
|--------|----------------------------------|
| Root   | The topmost node — has no parent |
| Leaf   | A node with no children          |
| Parent | The node directly above another  |
| Child  | A node directly below another    |

| Term           | Meaning                                                  |
|----------------|----------------------------------------------------------|
| <b>Sibling</b> | Nodes that share the same parent                         |
| <b>Subtree</b> | A node and all its descendants — treated as its own tree |
| <b>Depth</b>   | Distance from the ROOT to a node (root has depth 0)      |
| <b>Height</b>  | Longest path from a node DOWN to a leaf                  |
| <b>Level</b>   | Same as depth + 1 (root at level 1)                      |

## 2. Binary Tree — The Restriction That Matters

A BINARY TREE is a tree where every node has AT MOST TWO children, traditionally called LEFT and RIGHT.



Node 1 → left=2, right=3  
 Node 2 → left=4, right=5  
 Node 3 → left=null, right=6  
 Node 4 → left=null, right=null (leaf)

### Types of Binary Trees

| Type            | Definition                                                          |
|-----------------|---------------------------------------------------------------------|
| <b>Full</b>     | Every node has 0 or 2 children — never just 1                       |
| <b>Complete</b> | All levels filled except possibly the last, filled left to right    |
| <b>Perfect</b>  | All internal nodes have 2 children AND all leaves at the same depth |
| <b>Balanced</b> | Height of left and right subtrees differ by at most 1 at every node |
| <b>Skewed</b>   | All nodes have only one child — degenerate, behaves like a list     |

**Why "binary" matters:** Limiting each node to 2 children means we always have just LEFT and RIGHT to recurse into. That predictable structure is what makes tree algorithms clean — every problem decomposes into "do the work for the current node, then recurse into left and right."

### 3. Representing a Binary Tree in Java

A tree node holds a value and references to its two children. We define a `TreeNode` class and link nodes together with `new` — exactly like building a linked list, but branching.

#### The `TreeNode` Class

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}
```

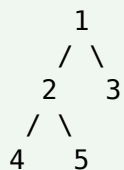
#### Building a Tree

```
// Build this tree:
//           1
//        /  \
//       2    3
//      /  \
//     4    5

TreeNode root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(3);
root.left.left = new TreeNode(4);
root.left.right = new TreeNode(5);
```

### 4. The Three Classic Traversals

A TRAVERSAL is the order in which we visit every node. For binary trees, there are three classic depth-first traversals — they differ only in WHEN they process the current node.



|           |                        |           |
|-----------|------------------------|-----------|
| Inorder   | (Left → Node → Right): | 4 2 5 1 3 |
| Preorder  | (Node → Left → Right): | 1 2 4 5 3 |
| Postorder | (Left → Right → Node): | 4 5 2 3 1 |

## 4.1 Inorder Traversal

```
public static void inorder(TreeNode node) {  
    if (node == null) return;  
    inorder(node.left);  
    System.out.print(node.val + " ");  
    inorder(node.right);  
}
```

For a Binary Search Tree, inorder visits nodes in SORTED order — that's its main superpower.

## 4.2 Preorder Traversal

```
public static void preorder(TreeNode node) {  
    if (node == null) return;  
    System.out.print(node.val + " ");  
    preorder(node.left);  
    preorder(node.right);  
}
```

Useful for COPYING a tree, since the root is visited first. Also used for prefix expressions.

## 4.3 Postorder Traversal

```
public static void postorder(TreeNode node) {  
    if (node == null) return;  
    postorder(node.left);  
    postorder(node.right);  
    System.out.print(node.val + " ");  
}
```

Useful for DELETING a tree (process children before deleting parent) and for computing values that depend on subtree results (like height).

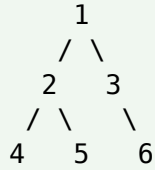
## 4.4 Memorizing the Difference

| Traversal | Order               | When useful                    |
|-----------|---------------------|--------------------------------|
| Inorder   | Left → Node → Right | Sorted output for BSTs         |
| Preorder  | Node → Left → Right | Copy or serialize a tree       |
| Postorder | Left → Right → Node | Delete a tree, compute heights |

**Mnemonic:** The names tell you WHERE the NODE is processed relative to the children. PRE-order = before the children. IN-order = between them (left, node, right). POST-order = after them. The recursive calls are always (left, right) — only the node print moves.

## 5. Level-Order Traversal (BFS)

The three traversals above are all DEPTH-FIRST — they go deep into the tree before coming back. There's also BREADTH-FIRST traversal, which visits the tree LEVEL BY LEVEL.



Level-order: 1 2 3 4 5 6  
(Level 0) (Level 1) (Level 2)

### Code Using a Queue

```
import java.util.LinkedList;
import java.util.Queue;

public static void levelOrder(TreeNode root) {
    if (root == null) return;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.val + " ");

        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}
```

**Depth-first vs Breadth-first:** DFS uses RECURSION (or an explicit stack). BFS uses a QUEUE. DFS is shorter to write but goes deep first. BFS naturally processes nodes in level order — perfect for shortest-path and level-related problems.

## 6. The Universal Tree Recursion Pattern

Almost every tree problem follows the same template. Once you internalize it, most problems become fill-in-the-blank.

```
ReturnType solve(TreeNode node) {
    if (node == null) return BASE_VALUE;    ← base case

    ReturnType left = solve(node.left);     ← solve left subtree
    ReturnType right = solve(node.right);   ← solve right subtree
```

```
        return COMBINE(node.val, left, right);    ← combine results
    }
```

Three steps every time:

1. Handle the null case — what should you return when the node doesn't exist?
2. Recurse into left and right — get answers for the two subtrees
3. Combine — use the current node's value plus the two subtree results to produce this node's answer

This pattern handles count problems, sum problems, height problems, validation problems — almost everything in the tree world.

## 7. Classic Tree Problems

### 7.1 Count Total Nodes

```
public static int countNodes(TreeNode node) {
    if (node == null) return 0;
    return 1 + countNodes(node.left) + countNodes(node.right);
}
```

Base case: empty tree has 0 nodes. Otherwise: 1 (this node) + count of left + count of right.

### 7.2 Sum of All Nodes

```
public static int sumNodes(TreeNode node) {
    if (node == null) return 0;
    return node.val + sumNodes(node.left) + sumNodes(node.right);
}
```

### 7.3 Find Maximum Value

```
public static int findMax(TreeNode node) {
    if (node == null) return Integer.MIN_VALUE;
    int leftMax = findMax(node.left);
    int rightMax = findMax(node.right);
    return Math.max(node.val, Math.max(leftMax, rightMax));
}
```

### 7.4 Height of the Tree

Height = longest path from a node down to a leaf. The empty tree has height -1, a leaf has height 0.

```
public static int height(TreeNode node) {
    if (node == null) return -1;
    int leftH = height(node.left);
    int rightH = height(node.right);
    return 1 + Math.max(leftH, rightH);
}
```

```
}
```

## 7.5 Search for a Value

```
public static boolean contains(TreeNode node, int target) {  
    if (node == null) return false;  
    if (node.val == target) return true;  
    return contains(node.left, target) || contains(node.right, target);  
}
```

## 7.6 Check if Two Trees Are Identical

```
public static boolean isSame(TreeNode a, TreeNode b) {  
    if (a == null && b == null) return true;  
    if (a == null || b == null) return false;  
    return a.val == b.val  
        && isSame(a.left, b.left)  
        && isSame(a.right, b.right);  
}
```

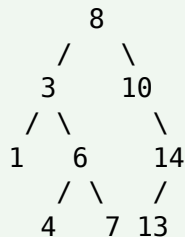
## 7.7 Invert a Tree (Mirror)

Famously rejected by Google: "We were going to hire you, but you can't invert a binary tree." — Max Howell (jest).

```
public static TreeNode invert(TreeNode node) {  
    if (node == null) return null;  
    TreeNode left = invert(node.left);  
    TreeNode right = invert(node.right);  
    node.left = right;  
    node.right = left;  
    return node;  
}
```

## 8. Introduction to Binary Search Trees (BST)

A BINARY SEARCH TREE is a binary tree with one extra rule: for every node, all values in the LEFT subtree are SMALLER, and all values in the RIGHT subtree are LARGER. This single rule unlocks  $O(\log n)$  search.



At every node: left subtree < node < right subtree

## 8.1 Search in a BST

```
public static boolean search(TreeNode node, int target) {
    if (node == null) return false;
    if (node.val == target) return true;
    if (target < node.val) return search(node.left, target);
    else return search(node.right, target);
}
```

Each step eliminates HALF the remaining tree. For a balanced BST, that's  $O(\log n)$  — same as binary search on an array, but a BST also supports fast inserts and deletes.

## 8.2 Insert into a BST

```
public static TreeNode insert(TreeNode node, int val) {
    if (node == null) return new TreeNode(val);
    if (val < node.val) node.left = insert(node.left, val);
    else if (val > node.val) node.right = insert(node.right, val);
    return node;
}
```

**Balance matters:** A BST is  $O(\log n)$  ONLY if it stays balanced. If you insert 1, 2, 3, 4, 5 in order, the tree becomes a straight line — degenerate into a linked list with  $O(n)$  search. Self-balancing trees (AVL, Red-Black) solve this. Java's `TreeMap/TreeSet` use Red-Black trees under the hood.

## 8.3 Validate a BST

Common interview question. A simple check that each node is greater than its left child and less than its right is NOT enough — you need range bounds.

```
public static boolean isValidBST(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return isValidBST(node.left, min, node.val)
        && isValidBST(node.right, node.val, max);
}

// Initial call:
isValidBST(root, Long.MIN_VALUE, Long.MAX_VALUE);
```

## 9. Time & Space Complexity

| Operation       | Avg (balanced) | Worst (skewed) | Space    |
|-----------------|----------------|----------------|----------|
| Traversal (any) | $O(n)$         | $O(n)$         | $O(h)^*$ |
| Search in BST   | $O(\log n)$    | $O(n)$         | $O(h)$   |



| Operation        | Avg (balanced) | Worst (skewed) | Space  |
|------------------|----------------|----------------|--------|
| Insert in BST    | $O(\log n)$    | $O(n)$         | $O(h)$ |
| Delete in BST    | $O(\log n)$    | $O(n)$         | $O(h)$ |
| Find min/max BST | $O(\log n)$    | $O(n)$         | $O(h)$ |

\*  $h$  = tree height. For balanced trees  $h = \log n$ ; for skewed trees  $h = n$ .

Key insight: recursive traversals use STACK SPACE equal to the height — not the total number of nodes. A balanced tree of a million nodes only uses ~20 levels of stack.

## 10. Common Mistakes

### Mistake 1: Missing the null check

```
public static void traverse(TreeNode node) {
    System.out.println(node.val);    // NullPointerException if node is null!
    traverse(node.left);
}

// Always start with:
if (node == null) return;
```

### Mistake 2: Confusing height with depth

Depth = distance from ROOT to a node. Height = distance from a node DOWN to deepest leaf. Easy to mix up — read the problem carefully.

### Mistake 3: Assuming a BST is automatically balanced

Insertion order matters. Insert 1, 2, 3, 4, 5 into a BST and you get a linked list —  $O(n)$  operations. For guaranteed balance use AVL, Red-Black, or just rely on Java's TreeMap/TreeSet.

### Mistake 4: Using == to compare values

`==` works for int, but if your tree holds Integer or other objects, use `.equals()`. For ints in `TreeNode.val`, `==` is fine.

### Mistake 5: Forgetting to return in recursive calls

```
if (val < node.val) {
    insert(node.left, val);    // x result is thrown away
}

if (val < node.val) {
    node.left = insert(node.left, val);    // ✓
```

```
}
```

### Mistake 6: Naive BST validation

Checking only that `left.val < node.val < right.val` is WRONG — you need range bounds that propagate through the whole subtree. The validate-BST mistake is one of the most common BST interview traps.

### Mistake 7: Modifying the tree while iterating

Just like with collections, modifying a tree mid-traversal can cause infinite loops or skipped nodes. Either collect first, then modify, or be very careful about traversal order.

## 11. Best Practices

- Always handle the null base case FIRST — it prevents most crashes
- Use the universal pattern: base case → recurse left → recurse right → combine
- For DFS, use recursion. For BFS, use a Queue (LinkedList or ArrayDeque)
- Trust the recursion — "if I assume left and right are solved correctly, what do I do here?"
- Draw the tree on paper before coding — small examples reveal the recursive pattern
- For BST validation, always pass min/max bounds, never compare only with immediate children
- Use Java's TreeMap or TreeSet when you need a balanced BST out of the box
- For very large trees, watch for stack overflow — convert to iterative with explicit stack/queue if needed

## 12. Quick Reference

| Concept                  | Meaning                                                                |
|--------------------------|------------------------------------------------------------------------|
| <b>TreeNode</b>          | Node class with val, left, right                                       |
| <b>root</b>              | The topmost node — entry point to the tree                             |
| <b>leaf</b>              | Node with no children                                                  |
| <b>height</b>            | Longest path from a node down to a leaf                                |
| <b>depth</b>             | Distance from root to a node                                           |
| <b>Inorder</b>           | Left → Node → Right (sorted for BST)                                   |
| <b>Preorder</b>          | Node → Left → Right (copy / serialize)                                 |
| <b>Postorder</b>         | Left → Right → Node (delete / compute height)                          |
| <b>Level-order (BFS)</b> | Visit nodes level by level (uses a queue)                              |
| <b>BST</b>               | Binary tree where <code>left &lt; node &lt; right</code> at every node |

| Concept                  | Meaning                                       |
|--------------------------|-----------------------------------------------|
| <b>Balanced</b>          | Heights of subtrees differ by at most 1       |
| <b>TreeMap / TreeSet</b> | Java's built-in balanced BST (Red-Black tree) |

## 13. Practice Problems

Mix of warm-ups, classics, and interview favorites.

### Warm-ups

- Build the tree shown in section 4 by hand. Print all three traversals.
- Count the total number of nodes in a binary tree.
- Compute the sum of all node values.
- Find the maximum value in a binary tree.
- Count the number of leaf nodes.
- Compute the height of a binary tree.

### Interview Classics

- Implement level-order traversal using a Queue. Print level by level.
- Check whether two binary trees are identical.
- Mirror (invert) a binary tree.
- Check if a binary tree is symmetric (mirror of itself).
- Find the diameter of a binary tree (longest path between any two nodes).
- Determine if a tree is height-balanced (heights of subtrees differ by at most 1).

### BST Problems

- Insert a sequence of integers into a BST and print inorder (should be sorted).
- Search for a value in a BST using the  $O(\log n)$  approach.
- Validate whether a given binary tree is a valid BST.
- Find the minimum and maximum value in a BST.
- Convert a sorted array into a balanced BST.
- Find the Kth smallest element in a BST.
- Find the Lowest Common Ancestor (LCA) of two nodes in a BST.

### Harder

- Find the LCA in a regular binary tree (not just BST).

24. Serialize and deserialize a binary tree (convert to/from string).
25. Print all root-to-leaf paths.
26. Maximum path sum (path can start and end at any nodes).

## 14. Final Takeaway

Binary trees take the recursion mindset you built in Chapter 29 and apply it to a structure that branches. Once you internalize the universal pattern — handle null, recurse left, recurse right, combine — most tree problems collapse from "impossible" to "obvious in 6 lines."

Don't try to trace big tree algorithms in your head. Trust the recursion. Assume the left and right subtrees give you correct answers, then ask: "what do I do at THIS node with those answers?" That single question solves the majority of tree problems.

**Key Takeaway:** Master the 5 fundamentals: count nodes, sum nodes, find max, compute height, search a value. Internalize the 3 traversals (inorder/preorder/postorder). Know the BST property. Practice the universal recursion pattern until it's automatic. Trees are 30-40% of typical coding interviews — this chapter is high-leverage.

## 15. What's Next?

Trees open the door to most advanced DSA topics:

- Tries — prefix trees for fast string lookup, autocomplete
- Heaps & Priority Queues — special complete binary trees for top-K problems
- AVL & Red-Black Trees — self-balancing BSTs
- N-ary Trees — generalization where nodes have more than 2 children
- Graph algorithms — DFS and BFS extend naturally from tree traversals
- Dynamic Programming on Trees — house robber III, diameter, max path sum
- Backtracking — N-Queens, Sudoku (search trees in disguise)
- Segment Trees / Fenwick Trees — for range query problems